Importing packages

eg → import math

math.sqrt(16)
⇒ 4
math.pi
⇒ 3.14159265
or → math.pow(2,3) ⇒ 8

→ import numpy as np

np.array([1,2,3,4])
⇒ array([1,2,3,4])

→ array must have homogeneous data → same data type.
→ memory efficient
→ faster operations

from math import *
sqrt(4)
⇒ 2

can import * to import all functions ✓

Custom packages

__init__.py

→ must be created to define packages and initialize their name spaces

Q How?

→ Create a folder name 'packages' in same directory.

→ make __init__.py.

→ Create modules inside packages.

Eg of libraries

(1) → import array.
arr = array.array ('i', [1,2,3,4])

⇒ arr = array ('i', [1,2,3,4])

↑ type code to restrict type of data in the array.

---

(2) import random

print (random.randint (1,10))

→ gives random int between 1 & 10

a = random.choice(['a', 'b', 'c'])

⇒ ~~a~~ returns a random choice ~~data~~ from given options.

---

(3) import os

os.listdir ('.')

os.mkdir ('new_dir')

print (os.getcwd ())

→ gives the current working directory

os.mkdir ('test_dir')

→ creates a new folder 'test_dir' in the current working directory.

PTO →

④ import shutil

shutil.copyfile('source.txt', 'destination.txt')

copies → new file made.

Data serialisation

⑤ import json

data = {'name': 'Krish', 'age': '25'}

json_str = json.dumps(data)

print(json_str) → {'name': Kr....

print(type(json_str)) → <class 'str'>

• parsed_data = json.loads(json_str)

→ converts data back into a dictionary.
→ type(parsed_data) ⟷ <class 'dict'>

---

⑥ import csv

new file made — write mode — newline

```
with open('example.csv', mode='w', newline='')
    as file:
    writer = csv.writer(file)
    writer.writerow(['name', 'age'])
    writer.writerow(['Krish', 25])
```

file identifier

⇒ ~~[name]~~
['name', 'age']
['Krish', 25]

```
with open ('example.csv', mode = 'r') as file:
    reader = csv.reader (file)
    for row in reader:
        print (row)
```

=> same output

---

(7) from datetime import datetime, timedelta

```
now = datetime.now()
```

→ gives the current date & time. ✓

```
yesterday = now - timedelta (days = 1)
```

→ gives the yesterdays date and time ✓

---

(8) import time

```
print (time.time())
```

↳ no idea what it prints?

```
time.sleep (2)
```

→ makes the interpreter/kernel sleep for 2 seconds ✓

print (time.time())

→ older time + 2.

(9) Regex (Regular expression)

```
import re
pattern = r'\d+'
text = 'There are 123 people in 256'
match = re.search(pattern, text)
print(match.group())
```

search this in this

returns the 1st ~~match~~ successful match in the given text.

⇒ 123

---

match function → used to match cases of input or other

```
user_input = input()
match user_input:
    case '1':
    case '2':
    case _:
```

↳ missing e.any matched cases